

Project Interim Report
On
**Online Course Management
System (Using Core Java & OOP
Concepts)**

RU-100-01-00012
Object Oriented Programming with Java



Submitted By
Shubham Kumar
ERP-RU-25-
11719

**Bachelor of Technology in Computer Science &
Engineering with AI and ML in association with IBM**

Under the Guidance of
Ashish shivhare

SCHOOL OF COMPUTER SCIENCE AND ENGINEERING

RUNGTA INTERNATIONAL SKILLS UNIVERSITY

BHILAI, CG

Abstract

The Online Course Management System is a console-based Java application developed to manage courses, student enrollments, and track learning progress. In today's digital age, e-learning platforms are widely used for education, and this project simulates a simplified version of such a system.

The system allows administrators to create and manage courses, register students, enroll them in courses, and monitor their progress. It is built entirely using Core Java and focuses on the practical application of Object-Oriented Programming (OOP) concepts including encapsulation, class interaction, method design, and array-based data management.

The project demonstrates how multiple interacting classes can be designed to model real-world entities such as students and courses, and how their relationships can be managed programmatically. The system is designed to be simple, clean, and efficient, making it suitable as an academic project to strengthen understanding of Java and OOP fundamentals.

1. Introduction :-

With the rapid growth of online education, managing courses and student data efficiently has become essential. Platforms like Coursera, NPTEL, and Udemy handle large volumes of student data along with course-wise progress tracking. This project is a simplified simulation of such a system, designed for educational purposes using Core Java.

The Online Course Management System is a console-based application that manages courses, registers students, handles enrollments, and tracks individual student progress in each enrolled course. The system is implemented in Java using fundamental OOP concepts and standard data structures such as arrays.

Objectives

- Design a system consisting of multiple interacting Java classes
- Manage course data including course name, ID, and duration
- Manage student data including name, ID, and learning progress
- Implement an enrollment mechanism allowing students to join courses
- Track and update student progress within enrolled courses
- Strengthen understanding of OOP principles in Java

Scope :-

This project is suitable as an academic demonstration of Java OOP concepts. It handles a limited dataset using arrays and can be extended in the future with dynamic storage (ArrayList), file handling, database connectivity, or a graphical user interface (GUI) using Java Swing.

2. System Requirements :-

Hardware Requirements

- Computer/Laptop with minimum 2 GB RAM
- Processor: Intel Core i3 or equivalent
- Storage: Minimum 500 MB free disk space

Software Requirements

- Operating System: Windows 10 / Linux / macOS
- Java Development Kit (JDK) 8 or higher
- IDE: VS Code / Eclipse / IntelliJ IDEA / EditPlus
- Terminal or Command Prompt to run the application

3. Methodology :-

The system was developed in a structured, step-by-step manner following the principles of Object-Oriented Programming. The methodology followed is described below:

Step 1: Identifying Classes

The system was analyzed and two primary entities were identified: Course and Student. Each entity was designed as a separate Java class with its own attributes (fields) and behaviors (methods).

Step 2: Designing the Course Class

The Course class was created with private fields: `courseName`, `courseId`, and `duration`. It also maintains an array of enrolled Student objects and exposes methods to add a student and display course details.

Step 3: Designing the Student Class

The Student class was created with private fields: `studentName`, `studentId`, and `progress`. It stores an array of enrolled Course objects. It provides an `enroll()` method to join a course, an `updateProgress()` method to update learning progress, and a `displayStudentDetails()` method.

Step 4: Implementing Enrollment Logic

The `enroll()` method in the Student class adds the given Course to the student's enrolled course array, and simultaneously calls the `addStudent()` method on the Course object. This establishes a two-way relationship between the student and the course.

Step 5: Progress Tracking

Each student has a single progress value (0-100) representing their overall learning completion. The `updateProgress()` method validates the input and updates this value accordingly. Progress is displayed whenever course or student details are printed.

Step 6: Testing via Main Class

The `CourseSystem` class serves as the main testing class. It creates three course objects and three student objects, performs enrollments, updates progress, and displays all course and student details.

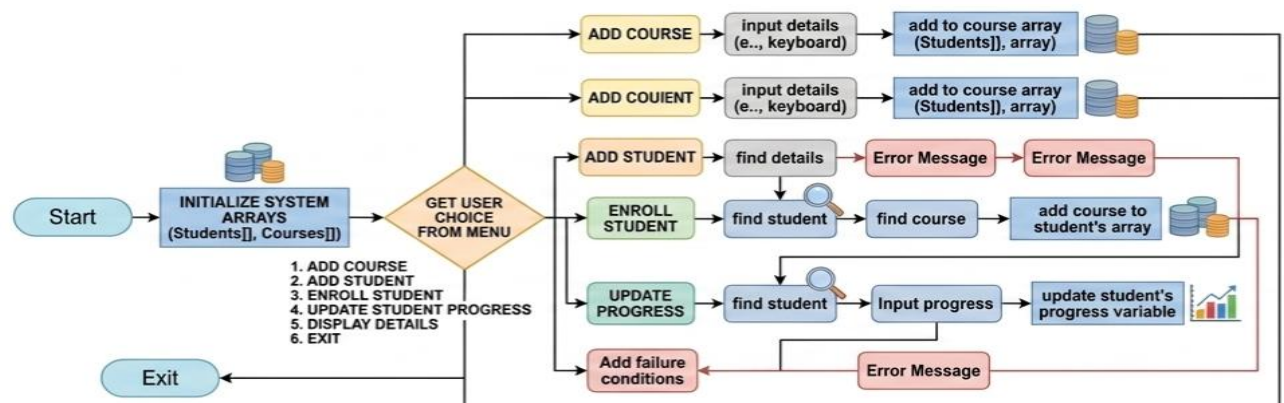
4. Flowchart and Class Diagram :-

System Flowchart

The overall flow of the system is as follows:

1. Program starts in the main method of CourseSystem class
2. Course objects are created with name, ID, and duration
3. Student objects are created with name and ID
4. Students are enrolled in courses using the enroll() method
5. Progress is updated for each student using updateProgress()
6. Course details are displayed — showing enrolled students and their progress
7. Student details are displayed — showing enrolled courses
8. Program ends

1. SYSTEM FLOWCHART



Class Diagram

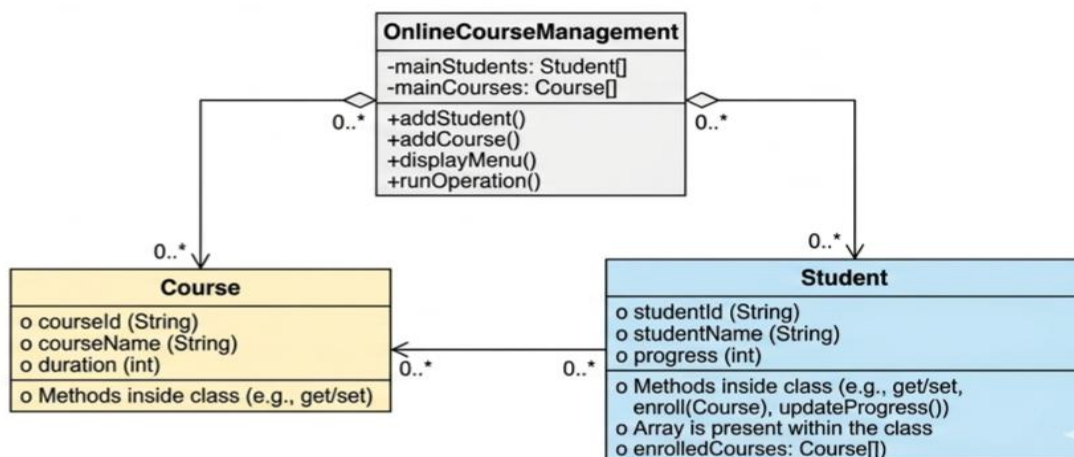
Course

- courseName: String
- courseId: String
- duration: String
- enrolledStudents: Student[]
- enrolledCount: int

+ Course(name, id, duration)
+ addStudent(student: Student): void
+ displayCourseDetails(): void
+ getCourseName(): String
+ getCourseId(): String

Student
- studentName: String - studentId: String - progress: int - enrolledCourses: Course[] - courseCount: int
+ Student(name, id) + enroll(course: Course): void + updateProgress(value: int): void + displayStudentDetails(): void + getProgress(): int

2. SYSTEM CLASS DIAGRAM (Representing System Structure & Interaction)



5. Implementation :-

Course Class

The Course class is responsible for representing a course in the system. It holds three private fields: `courseName`, `courseId`, and `duration`. It also maintains a fixed-size array of Student objects (maximum 10 students) along with a counter to track the number of enrolled students. The `addStudent()` method appends a student to this array, and the `displayCourseDetails()` method prints all course information along with each enrolled student's name and current progress.

Student Class

The Student class models a student in the system. It stores the student's name, ID, and their progress value as private fields. A fixed-size array of Course objects (maximum 5) tracks the courses the student has joined. The `enroll()` method both adds the course to the student's list and notifies the Course object by calling its `addStudent()` method, establishing a bidirectional link. The `updateProgress()` method validates the input and sets the new progress value.

CourseSystem Class (Main)

The CourseSystem class contains the main() method and serves as the driver of the program. It instantiates three courses (Java Programming, Data Structures, Web Development) and three students (Rahul, Priya, Amit). Various enrollments are performed, progress is updated for each student, and then the full details of all courses and students are displayed on the console.

Sample Output

The following output was produced when the program was compiled and executed:

===== ENROLLING STUDENTS IN COURSES =====

Rahul has been successfully enrolled in: Java Programming
Rahul has been successfully enrolled in: Data Structures
Priya has been successfully enrolled in: Java Programming
Priya has been successfully enrolled in: Web Development
Amit has been successfully enrolled in: Data Structures
Amit has been successfully enrolled in: Web Development

===== UPDATING STUDENT PROGRESS =====

Progress updated for Rahul: 80%
Progress updated for Priya: 65%
Progress updated for Amit: 50%

===== COURSE DETAILS =====

=====

Course Name : Java Programming
Course ID : C101
Duration : 3 Months
Enrolled Students:
1. Rahul - Progress: 80%
2. Priya - Progress: 65%

=====

=====

Course Name : Data Structures
Course ID : C102
Duration : 2 Months
Enrolled Students:
1. Rahul - Progress: 80%

2. Amit - Progress: 50%

Course Name : Web Development

Course ID : C103

Duration : 4 Months

Enrolled Students:

1. Priya - Progress: 65%

2. Amit - Progress: 50%

===== STUDENT DETAILS =====

Student Name : Rahul

Student ID : S001

Progress : 80%

Enrolled Courses:

1. Java Programming (ID: C101)

2. Data Structures (ID: C102)

Student Name : Priya

Student ID : S002

Progress : 65%

Enrolled Courses:

1. Java Programming (ID: C101)

2. Web Development (ID: C103)

Student Name : Amit

Student ID : S003

Progress : 50%

Enrolled Courses:

1. Data Structures (ID: C102)

2. Web Development (ID: C103)

6. Gantt Chart :-

The following Gantt chart represents the timeline followed during the development of this project:

Task	Week 1	Week 2	Week 3	Week 4
Requirement Analysis & Planning	✓			
Class Design (Course & Student)	✓	✓		
Coding & Implementation		✓	✓	
Testing & Debugging			✓	✓
Report Writing & Documentation				✓

7. Future Enhancements :-

The current implementation uses static arrays and a console interface. The following enhancements can be made to improve the system:

- Replace arrays with ArrayList for dynamic and flexible storage of students and courses
- Add a menu-driven interface to allow interactive user input at runtime
- Implement search functionality to find a student or course by name or ID
- Support multiple progress values per course (per-module tracking)
- Add file handling to save and load data persistently across program runs
- Develop a graphical user interface (GUI) using Java Swing for better usability
- Integrate with a database (MySQL/SQLite) using JDBC for large-scale data storage

8. Conclusion :-

The Online Course Management System was successfully developed using Core Java and Object-Oriented Programming concepts. The project demonstrates how real-world entities such as students and courses can be modeled using Java classes, and how their interactions (enrollment, progress tracking) can be implemented through well-designed methods.

Through this project, key OOP concepts such as encapsulation (private fields with

getter methods), object interaction (students and courses referencing each other), and method design were practically applied. Arrays were used as the primary data structure to store student and course collections.

The system fulfills all the objectives defined at the beginning — it manages courses, registers students, allows enrollment, tracks progress, and displays information clearly. The project serves as a strong foundation that can be extended further with dynamic data structures, persistent storage, and a richer user interface in the future.

References

- [1] H. Schildt, **Java: The Complete Reference**, 11th ed. New York: McGraw-Hill, 2018.
- [2] Oracle, "Java Documentation," Oracle. [Online]. Available: <https://docs.oracle.com>
- [3] W3Schools, "Java OOP," W3Schools. [Online]. Available: https://www.w3schools.com/java/java_oop.asp
- [4] GeeksforGeeks, "Object Oriented Programming in Java," GeeksforGeeks. [Online]. Available: <https://www.geeksforgeeks.org/object-oriented-programming-oops-concept-in-java/>